

Self-Play Meets Skill Evolution: Self-Evolving Search Agents that Pose, Solve, and Remember

Zenghuang Fu^{1,2*}, Zhaoyang Li^{3*}, Qiuyuan Ai^{3*}, Haoyu Wu³,
Minghui Wu⁴, Chenxu Zhao⁴, Ante Wang⁵, Guannan He^{3†}, Changwei Wang^{6,7†}

¹University of Chinese Academy of Sciences

²Institute of Automation, Chinese Academy of Sciences

³Peking University

⁴Mininglamp Technology

⁵Tsinghua University

⁶Key Laboratory of Computing Power Network and Information Security, Ministry of Education; Shandong Computer Science Center, Qilu University of Technology (Shandong Academy of Sciences)

⁷Key Laboratory of Computing Power Internet and Service Computing, Shandong Fundamental Research Center for Computer Science

Abstract

Self-play agents can generate training problems without questions from target benchmarks, but their curricula lack persistent state: failures affect gradients yet do not explicitly shape future practice. External skill memories preserve procedural experience but are typically learned from fixed task distributions. We introduce **SESA** (Self-Evolving Skill-Augmented Agent), which makes procedural memory an evolving state of tool-augmented search self-play. A challenger poses problems, while a separately parameterized solver alone retrieves skills. Informative failures are distilled into reusable skills and written back to memory. The updated memory changes solver behavior and success, which changes the challenger’s reward and the distribution of future problems; the resulting frontier produces new failures that rewrite memory. This bidirectional loop makes task generation and skill memory co-evolve. Because retrieved skills shape on-policy training trajectories, their benefits can enter the model parameters as well as remain in the external bank, enabling memory-free deployment and optional inference-time retrieval. Across seven open-domain and multi-hop question-answering benchmarks, SESA improves average accuracy over SSP by 1.2–3.2 points across multiple backbones and surpasses the skill-augmented SkillRL baseline by 0.9 points under a unified evaluation protocol. On Qwen3 models, SESA-Off retains 1.8–2.2 points of improvement over SSP, while the final skill bank adds a further 0.5–1.0 points. These results show that evolving skill memory is not merely an inference-time plug-in: it changes policy learning and the future training distribution while retaining value as optional external memory. Our code is available at <https://github.com/Zenghuang-Fu/SESA-Self-Evolving-Search-Agents>.

Introduction

Large language model (LLM) agents are increasingly trained from their own experience rather than a fixed corpus of demonstrations. One route is *zero-data self-play*, where an agent poses and solves problems using verifiable rewards

*These authors contributed equally.

†Corresponding author.

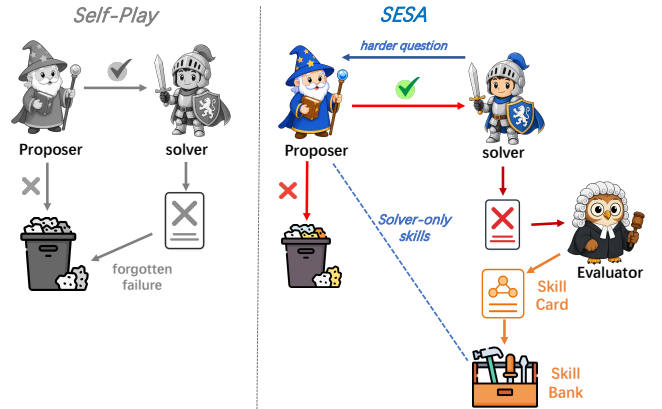


Figure 1: Conceptual overview of SESA. Self-posed search failures are not discarded: they are distilled into reusable skills, stored in memory, and fed back to the solver so the next self-play round moves toward harder problems.

without an external question set (Lu et al. 2026; Chen et al. 2025; Xia et al. 2025; Acikgoz et al. 2026). Because task generation is endogenous, self-play can adapt difficulty as the solver improves. Yet its experience is usually transient: a trajectory contributes a policy gradient but leaves no explicit, reusable account of the strategy learned from it.

A complementary route, *skill-augmented reinforcement learning*, distills experience into retrievable strategies or structured notes (Xia et al. 2026; Wang et al. 2026; Shi et al. 2026; Li et al. 2026a,b; Zhang et al. 2026). Such memories preserve procedural knowledge beyond the update that produced it, but they are typically built from fixed datasets or hand-designed curricula. These two limitations are mirror images: self-play decides what to practice but forgets its lessons, whereas skill-augmented RL remembers lessons

from tasks it did not choose. The missing setting is an agent that can do both.

We present **SESA** (Self-Evolving Skill-Augmented Agent), which places online skill evolution inside tool-augmented search self-play. A challenger poses a question with a verifiable target, and a separately parameterized solver attempts it with a search tool. Informative solver failures are distilled into human-readable skills, deduplicated, and written to a bounded non-parametric memory. The solver retrieves these skills in later rounds, so past failures change future on-policy trajectories and the data used for policy optimization. Memory remains hidden from the challenger, preventing direct skill leakage and preserving an asymmetric game between problem generation and problem solving.

Simply composing self-play and a skill bank is not sufficient. Storing every failure would accumulate noise and redundancy; exposing the same memory to the challenger could leak solution strategies into generated questions; and adding retrieval only after training would not change the self-play curriculum. A closed loop must therefore decide which failures are learnable, who may access their distilled lessons, and how those lessons return to on-policy training.

SESA realizes this feedback loop through four stages. *Memory priming* provides an initial retrieval substrate; *asymmetric self-play* gives solver-only access to skills; *frontier shaping* steers the challenger toward problems near the solver’s current competence boundary; and *failure distillation* converts useful failed rollouts into new skills. The result is a failure-to-skill-to-solver loop: self-posed problems expose weaknesses, those weaknesses become reusable guidance, and the strengthened solver pushes the challenger toward a new frontier.

Because skills participate during training, SESA supports two forms of reuse. Skill-conditioned rollouts can leave *parametric carryover* in the trained solver, allowing memory-free inference. The final bank can also remain enabled for additional *non-parametric augmentation*. We isolate these effects by comparing SSP, SESA with memory disabled (SESA-Off), and the same trained SESA solver with memory enabled (SESA-On). This distinction shows whether skill gains reside in the policy, the external bank, or both.

We evaluate on 3,125 held-out questions spanning seven factual and multi-hop search benchmarks. In the completed runs, SESA-On improves average accuracy over SSP by 2.3 points on Qwen3-4B, 3.2 points on Qwen3-8B, and 1.2 points on LLaMA-3.1-8B, while exceeding the corresponding base models by 10.9, 7.0, and 10.8 points. These gains across model scale and family indicate that persistent skill evolution adds value beyond self-play alone; the controlled Off/On comparison further tests how much of that value is parametric.

We make three contributions.

- **Coupled self-evolution.** We unite self-posed problem generation with persistent skill consolidation, enabling an agent to choose its practice frontier and retain lessons from its own failures.
- **The SESA loop.** Solver-only retrieval, frontier shaping, and online failure distillation feed reusable skills back

into subsequent self-play without leaking memory to the challenger.

- **Dual-path evaluation.** We separate memory-free parametric carryover from inference-time retrieval gains across multiple search benchmarks and model families.

Related Work

Self-Play for Agent Training

Self-play has become a practical way to train agents without a fixed pool of human-written tasks. Search Self-Play (SSP) (Lu et al. 2026) trains proposer and solver policies for retrieval-augmented search using only a verifiable reward; Multi-Agent Evolve (Chen et al. 2025) extends this idea with a proposer–solver–judge game; Tool-R0 (Acikgoz et al. 2026) studies zero-data self-play for tool use; and EvolveR (Wu et al. 2025) frames self-evolution as an experience-driven lifecycle. R-Few (Yu et al. 2025) and Agent0 (Xia et al. 2025) further emphasize asymmetric roles and adaptive difficulty. These methods make task generation endogenous, but the solving experience is usually consumed as a training trajectory and then discarded. SESA follows the self-posed setting but adds an explicit consolidation path: failed solver rollouts become retrievable skills that affect later self-play rounds.

Skill Memory and Experience Consolidation

A complementary line studies agents that store reusable experience outside the model weights (Ai et al. 2026). SkillRL (Xia et al. 2026) combines a cold-start skill bank, retained failures, and policy–skill co-evolution; related work further studies reinforcement learning with skill libraries (Wang et al. 2026), as well as skill rollout, selection, structure, and co-evolution in Skill1 (Shi et al. 2026), SkillGraph (Li et al. 2026a), ARISE (Li et al. 2026b), and CoEvoSkills (Zhang et al. 2026). Other systems focus on skill curation and lifecycle management (Ouyang et al. 2026; Pu, Song, and Zhao 2026; Lin et al. 2026b,a), while Voyager (Wang et al. 2024), Reflexion (Shinn et al. 2023), and ExpeL (Zhao et al. 2024) show that non-parametric memories can make agent experience reusable and inspectable. However, these systems typically learn skills from fixed datasets, hand-designed curricula, or non-RL interaction loops. SESA differs by placing skill consolidation inside zero-data self-play: the agent both creates the search problems that expose failures and writes those failures back into a memory that changes future solving behavior. The distinction from SkillRL is therefore not merely whether a skill bank is present. SkillRL evolves skills under an exogenous task distribution, whereas SESA lets the solver’s evolving memory change its behavior on an endogenous frontier, which in turn changes the reward that trains the challenger. Task generation and procedural memory thus become coupled parts of the same learning process.

Method

Setup and Notation

SESA trains a tool-augmented search agent through self-play, with no external question set. A *proposer* (challenger) policy

π_p generates a search problem; a *solver* (learner) policy π_s attempts it by issuing retrieval queries to a fixed search tool and producing a final answer; and a verifiable reward compares the answer against the proposer-provided target. On top of this self-play backbone, SESA maintains a non-parametric *skill memory* $\mathcal{B}$: a set of retrievable, human-readable strategies that the solver consults during training and can optionally retain at inference time, and that grows from the solver’s own failures. Each skill is stored as

$$s = (u, c, a, z, m), \quad (1)$$

where u is its description, c the trigger conditions, a avoidance cues (anti-patterns or common confusions), z reusable query templates, and m the retrieval, helpfulness, and hurt counts used for maintenance. The training loop is organized into four algorithmic stages: *memory priming*, *asymmetric self-play*, *frontier shaping*, and *failure distillation*. Figure 2 shows how these stages close the SESA flywheel. We first define the agentic reinforcement-learning objective that drives the self-play game, then describe each stage below and analyze why they must couple in the next section.

Memory Priming

SESA initializes $\mathcal{B}$ with 15 hand-written skills covering recurring search patterns and 142 deduplicated skills mined during an earlier self-play bootstrap:

$$\mathcal{B}_0 = \mathcal{B}_{\text{seed}} \cup \mathcal{B}_{\text{warm}}. \quad (2)$$

Across the 157 initial entries, descriptions, triggers, avoidance cues, query templates, and usage metadata provide an initial retrieval substrate and anchor the granularity of later skill distillation and deduplication.

Asymmetric Self-Play

SESA uses separately parameterized proposer and solver policies, following asymmetric self-play (Xia et al. 2025). This separation gives the solver’s success rate a stable interpretation as the difficulty of proposer-generated problems and, crucially, lets SESA expose retrieved skills only to the solver. The proposer therefore adapts through reward feedback without directly observing solution-oriented memory. Additional motivation and implementation details for this information asymmetry appear in the supplementary material.

Agentic RL Objective

SESA optimizes both roles with a critic-free policy-gradient backend built on Group Relative Policy Optimization (GRPO). The solver uses the standard grouped form: for each generated problem $x = (q, a^*)$, it samples G independent search rollouts

$$\tau_i \sim \pi_s(\cdot \mid q, \mathcal{T}, R(q; \mathcal{B}_t)), \quad i = 1, \dots, G, \quad (3)$$

where $\mathcal{T}$ is the search tool and $R(q; \mathcal{B}_t)$ is the solver-only retrieved skill context. Each rollout produces a final answer $\hat{a}_i$. The solver receives a verifiable answer reward,

$$r_s(\tau_i, a^*) = \mathbf{1}\{\text{Judge}(\hat{a}_i, a^*) = 1\}, \quad (4)$$

where the judge first checks exact match after normalization and otherwise uses a model-based semantic match against the

target answer. This reward is sparse but reliable, and is assigned to the terminal response token. Because the retrieved context $R(q; \mathcal{B}_t)$ enters the *on-policy* rollouts in Eq. (1), skills do not merely condition a single inference pass: they reshape the trajectory distribution on which the policy gradient is computed. Retrieved guidance is thus internalized into the solver parameters during training rather than acting only as an inference-time prompt, which is why the trained solver retains most of its advantage even with the bank disabled (cf. Table 2).

The proposer is optimized against a difficulty-shaped reward derived from the same solver group (defined in the next section), so that it learns to pose problems near the solver’s competence frontier. For solver updates, GRPO normalizes rewards within the G rollouts of the same problem,

$$\hat{A}_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^G)}{\text{std}(\{R_j\}_{j=1}^G) + \epsilon}, \quad (5)$$

and applies the resulting advantage to the generated response tokens. The proposer emits one candidate per training instance and obtains its learning signal only after the solver group has been evaluated, and is updated with the same critic-free policy-gradient backend.

Frontier Shaping

Not every self-posed problem yields a useful gradient, and not every failure yields a useful skill. Problems the solver always gets right carry no learning signal; problems it always gets wrong are usually noise—outside the solver’s current reach—and the failures they produce, if consolidated, would pollute the skill memory with un-actionable entries. SESA therefore keeps learning on problems near the solver’s competence boundary, using the same empirical difficulty signal that the solver group already provides,

$$\hat{p}_s(x) = \frac{1}{G} \sum_{i=1}^G \mathbf{1}\{r_s(\tau_i, a^*) > 0\}, \quad (6)$$

the fraction of the G solver rollouts that succeed on problem x .

Rather than hard-filtering batches, SESA shapes the proposer’s reward so that the challenger is trained to *generate* frontier-difficulty problems in the first place. A naïve complement-of-accuracy reward $1 - \hat{p}_s(x)$ encourages harder questions but over-rewards unsolvable ones and destabilizes self-play. SESA instead uses a bell-shaped, endpoint-penalized reward,

$$r_p(x) = \begin{cases} -\lambda, & \hat{p}_s(x) \in \{0, 1\}, \\ 4(\ell + \hat{p}_s(x))(h - \hat{p}_s(x)), & \text{otherwise,} \end{cases} \quad (7)$$

with $\ell = 0$, $h = 1$, and endpoint penalty $\lambda > 0$. The reward peaks at intermediate success rates and penalizes both trivial ($\hat{p}_s = 1$) and unsolvable ($\hat{p}_s = 0$) questions, so the proposer is continually pushed to pose problems just beyond the solver’s current ability. As the solver improves, the same reward drives the challenger toward a new frontier. Failures sampled from this shaped distribution then flow into skill distillation, ensuring the memory receives exploitable rather than noisy signal.

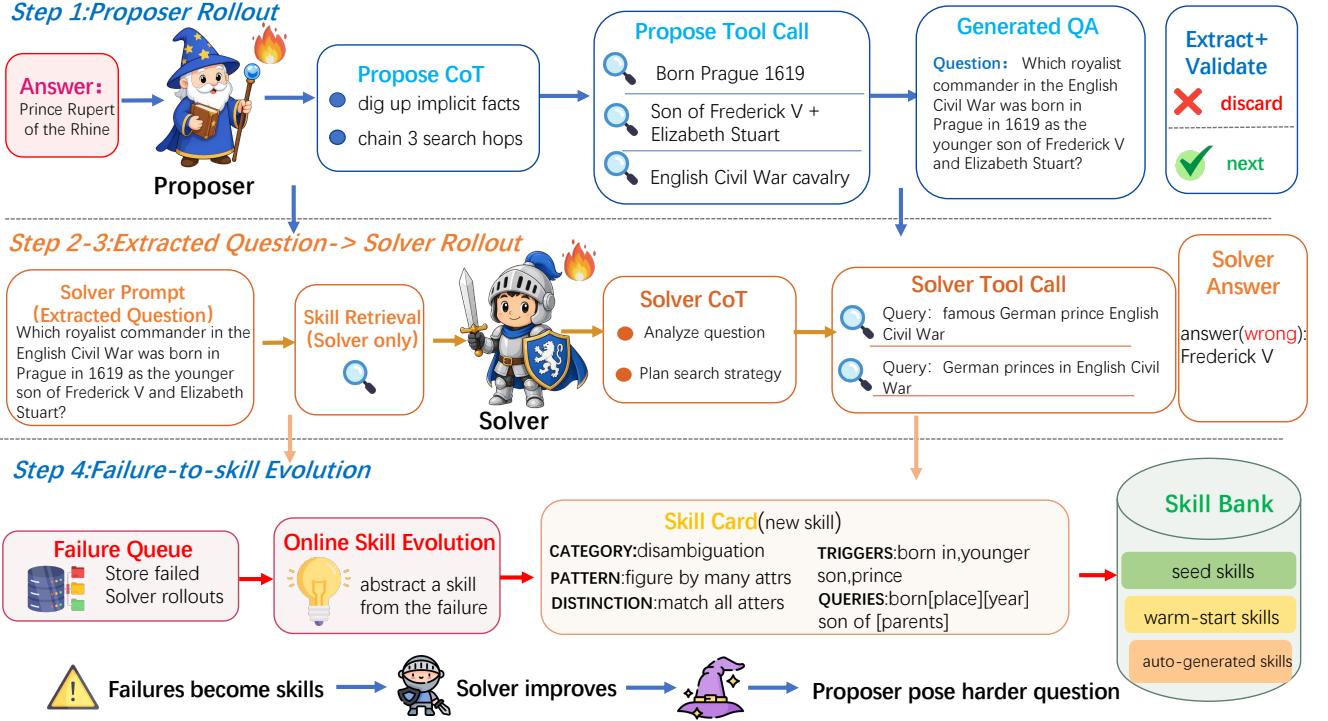


Figure 2: The SESA training loop. Memory priming seeds a retrievable skill bank; asymmetric self-play lets a challenger pose search tasks while only the solver can retrieve skills; frontier shaping steers the challenger toward problems near the solver’s solvability boundary; and failure distillation converts failed rollouts into deduplicated skills that are written back to memory. The updated bank strengthens the solver and raises the frontier for subsequent challenger-generated problems.

Failure Distillation

Failure distillation turns transient failures into durable, retrievable knowledge; it is what the self-play line lacks. It runs as a three-phase lifecycle synchronized with each training step.

Retrieval. Before the solver attempts a problem, SESA embeds the question and retrieves the top- k most similar skills from $\mathcal{B}$ using a dense encoder,

$$R(q; \mathcal{B}_t) = \text{TopK}_{s \in \mathcal{B}_t} \text{sim}(e(q), e(s)), \quad (8)$$

and prepends them to the solver’s prompt as reference strategies. Retrieval is read-only and deterministic given the current bank, and—by asymmetric self-play—happens for the solver only.

Failure collection. After reward computation, failed solver rollouts are summarized into compact records containing the problem, target, retrieved evidence, prediction, and retrieved skill identifiers:

$$\mathcal{F}_t = \{(q, a^*, \hat{a}, R(q; \mathcal{B}_t)) \mid r(\hat{a}, a^*) = 0\}. \quad (9)$$

Only informative frontier failures enter a 300-record pending queue. Every 10 steps, once at least 20 have accumulated, consolidation selects at most 30, prioritizing repeated failures and those unsolved despite retrieved guidance.

Consolidation. A judge abstracts each selected failure into its trigger, distinguishing evidence, avoidance cues, and query templates, targeting what prior guidance missed. A candidate is admitted only if its E5-base-v2 cosine similarity is at most 0.93 against the bank and candidates already admitted in the same update:

$$\max_{s' \in \mathcal{B}_t \cup \Delta \mathcal{B}_t^{<s}} \text{sim}(e(s), e(s')) \leq 0.93. \quad (10)$$

After admission, the maintained bank becomes

$$\mathcal{B}_{t+1} = \text{Maintain}(\mathcal{B}_t \cup \Delta \mathcal{B}_t). \quad (11)$$

Seed skills are retained. A non-seed skill is evicted after at least three retrievals if its helpful count minus hurt count is negative; overflow beyond 800 entries removes the lowest-scoring non-seed skills. At eligible step boundaries, the trainer launches at most one consolidation job asynchronously. Completion persists and increments the bank version seen by later retrievals. Utility is assigned from the same solver rollouts: a correct answer increments helpfulness for each retrieved skill, whereas a substantive incorrect answer increments hurt; malformed trajectories are ignored. Retention therefore reflects observed downstream behavior rather than age alone, and step-boundary commits prevent the bank from changing within the solver batch that produced the evidence.

Over training, this lifecycle makes the skill memory a living object: it grows where the solver fails, forgets what

does not help, and—because the failures come from self-posed problems that get harder as the solver improves—keeps acquiring skills for a difficulty frontier that no fixed dataset defines.

Dual-Path Skill Reuse

Because retrieved skills shape the on-policy trajectories used for training, SESA can transfer experience through both model parameters and external memory. Let θ_T and $\mathcal{B}_T$ denote the trained solver and final bank. Memory-free deployment (SESA-OFF) uses

$$\pi_{\text{off}}(\tau \mid q) = \pi_{\theta_T}(\tau \mid q, \mathcal{T}), \quad (12)$$

whereas memory-augmented deployment (SESA-ON) uses

$$\pi_{\text{on}}(\tau \mid q) = \pi_{\theta_T}(\tau \mid q, \mathcal{T}, R(q; \mathcal{B}_T)). \quad (13)$$

Comparing SSP with SESA-OFF measures parametric carryover; comparing SESA-OFF with SESA-ON isolates inference-time retrieval. Because SESA has no explicit skill-distillation loss, carryover is an empirical outcome rather than an architectural assumption. Further discussion is provided in the supplementary material.

Experiments

Experimental Setup

Training data. SESA follows the zero-data setting of SSP: training does not consume questions from any evaluation benchmark. We use the released SSP pool of 50,000 target answers paired with one-, two-, or three-hop requirements (16,547/16,729/16,724 seeds). Preprocessing inserts each pair into the challenger prompt without revealing the target. At every iteration, the challenger must use the search engine to turn a sampled seed into a concise, uniquely answerable question whose solution requires the specified number of hops. The solver then attempts the generated question, and informative failures produced under the frontier-shaped objective enter the skill-distillation queue. This separation ensures that improvements on the test benchmarks reflect transfer from self-generated search experience rather than supervised exposure to their questions.

Evaluation datasets. We evaluate on 3,125 held-out questions from seven benchmarks. Natural Questions (NQ) (Kwiatkowski et al. 2019), TriviaQA (Joshi et al. 2017), and PopQA (Mallen et al. 2023) primarily test open-domain factual retrieval; HotpotQA (Yang et al. 2018), 2WikiMultiHopQA (2Wiki) (Ho et al. 2020), and MuSiQue (Trivedi et al. 2022) emphasize compositional multi-hop search; and Bamboogle (Press et al. 2023) provides a compact, challenging set of 125 questions that are difficult to answer without explicit decomposition. We use 500 examples from each of the first six datasets and all 125 Bamboogle examples. This mix tests whether the skills learned from self-posed problems transfer across both fact-oriented and multi-hop distributions.

Models and baselines. We study Qwen3-4B, Qwen3-4B-Instruct, and Qwen3-8B (Yang et al. 2025); Qwen2.5-7B-Base and Qwen2.5-7B-Instruct (Qwen et al. 2025); LLaMA-3.1-8B (Grattafiori et al. 2024); and the search-specialized

Search-R1-7B (Jin et al. 2025). For each backbone, *Base* denotes the pretrained checkpoint before continual training, *SSP* denotes self-posed self-play (Lu et al. 2026) without a skill bank, and *SESA* adds the closed-loop skill evolution described in our method. Base, SSP, and SESA use the same search backend and answer format. SSP and SESA are trained from the same corresponding initialization; SESA differs only in the skill path unless stated otherwise.

Metrics and evaluation protocol. The main metric is answer accuracy (%), averaged equally over the seven dataset-level scores. We first apply normalized exact match; predictions that do not match lexically are checked for semantic equivalence by Qwen2.5-32B-Instruct (Qwen et al. 2025). We use greedy decoding with one rollout per question and at most 10 assistant/search turns. Exact match and token-level F1 are retained as diagnostic metrics but are not mixed into the main-table average. Unless stated otherwise, the main table reports SESA-On.

Implementation details. We train with GRPO using five solver rollouts per generated problem. Retrieval returns the top three E5-base-v2 records, and DeepSeek-v4-pro performs skill distillation. Training uses $8 \times$ NVIDIA A100-SXM4-80GB GPUs. Full optimization, sequence-length, and distributed-training settings appear in the supplementary material.

Main Results

Table 1 compares each base search agent with SSP and SESA. This layout separates the gain from self-play itself (Base to SSP) from the additional gain of making self-play experience persistent (SSP to SESA), while holding the backbone fixed.

Persistent skills improve self-play across scales. On the three Qwen3 backbones, SESA improves average accuracy over SSP by 2.3 points on Qwen3-4B, 2.7 points on Qwen3-4B-Instruct, and 3.2 points on Qwen3-8B. The corresponding gains over the untrained checkpoints are 10.9, 11.8, and 7.0 points. The advantage therefore does not vanish as the backbone grows or after instruction tuning: self-play provides the first improvement, while persistent failure consolidation adds a further gain. The same trend appears on the two Qwen2.5 settings, where SESA exceeds SSP by 2.1 and 1.5 average points.

The gain transfers across model families. On LLaMA-3.1-8B, SESA reaches 47.5 average accuracy, improving over SSP by 1.2 points and over the base model by 10.8 points. On the search-specialized Search-R1-7B initialization, it reaches 57.5 and remains above SSP, showing that the skill loop still contributes after search-oriented training. Improvements are not uniform at the dataset level: Qwen3-4B is slightly below SSP on 2Wiki, and Qwen3-8B is lower on Bamboogle. We therefore interpret SESA as a consistent average improvement across initializations, rather than a guarantee of monotonic gains on every benchmark.

Where Do Skill Gains Reside?

To separate improvements carried by the trained policy from improvements that require retrieval at inference time, we

Method	NQ	TriviaQA	PopQA	HotpotQA	2Wiki	MuSiQue	Bamboogle	Avg.
Continual Training on Qwen3 Backbones								
Qwen3-4B	46.4	65.8	45.0	42.8	43.0	20.0	54.4	45.3
+ SSP	54.4	76.8	53.6	56.4	52.8	25.4	57.6	53.9
+ SESA	56.2 ^{+9.8}	80.4 ^{+14.6}	55.2 ^{+10.2}	57.8 ^{+15.0}	51.8 ^{+8.8}	27.2 ^{+7.2}	64.8 ^{+10.4}	56.2 ^{+10.9}
Qwen3-4B-Instruct	48.8	71.8	42.6	52.0	35.6	21.8	50.4	46.1
+ SSP	57.6	75.4	51.6	59.2	49.8	28.6	64.0	55.2
+ SESA	63.6 ^{+14.8}	80.4 ^{+8.6}	56.0 ^{+13.4}	59.2 ^{+7.2}	54.0 ^{+18.4}	28.6 ^{+6.8}	63.2 ^{+12.8}	57.9 ^{+11.8}
Qwen3-8B	53.6	76.0	50.8	54.2	48.0	26.6	58.4	52.5
+ SSP	56.0	78.2	55.0	58.0	51.5	28.0	67.2	56.3
+ SESA	62.2 ^{+8.6}	82.8 ^{+6.8}	57.0 ^{+6.2}	64.0 ^{+9.8}	54.6 ^{+6.6}	32.6 ^{+6.0}	63.2 ^{+4.8}	59.5 ^{+7.0}
Continual Training on Qwen2.5 Backbones								
Qwen2.5-7B-Base	32.0	33.2	25.0	18.0	10.8	11.0	26.4	22.3
+ SSP	54.2	73.6	56.0	52.8	33.2	24.0	47.2	48.7
+ SESA	58.8 ^{+26.8}	72.2 ^{+39.0}	61.4 ^{+36.4}	53.8 ^{+35.8}	38.0 ^{+27.2}	26.0 ^{+15.0}	45.6 ^{+19.2}	50.8 ^{+28.5}
Qwen2.5-7B-Instruct	44.2	64.0	36.4	45.0	32.8	16.8	51.2	41.5
+ SSP	54.8	73.4	51.8	51.8	38.8	21.2	54.4	49.5
+ SESA	57.4 ^{+13.2}	72.2 ^{+8.2}	55.2 ^{+18.8}	52.0 ^{+7.0}	42.2 ^{+9.4}	27.0 ^{+10.2}	51.2 ^{+0.0}	51.0 ^{+9.5}
Continual Training on Cross-Family Backbones								
LLaMA-3.1-8B	50.2	65.2	45.8	34.6	19.4	11.4	30.4	36.7
+ SSP	58.0	75.8	55.4	44.2	34.4	16.2	40.0	46.3
+ SESA	61.2 ^{+11.0}	79.2 ^{+14.0}	55.2 ^{+9.4}	47.0 ^{+12.4}	35.2 ^{+15.8}	15.2 ^{+3.8}	39.2 ^{+8.8}	47.5 ^{+10.8}
Continual Training on Search-Specialized Agents								
Search-R1-7B	56.6	75.4	57.2	58.2	45.2	29.6	55.2	53.9
+ SSP	57.8	78.0	58.4	60.4	45.6	30.6	59.2	55.7
+ SESA	63.0 ^{+6.4}	80.4 ^{+5.0}	60.8 ^{+3.6}	62.8 ^{+4.6}	45.6 ^{+0.4}	32.0 ^{+2.4}	57.6 ^{+2.4}	57.5 ^{+3.6}

Table 1: Answer accuracy (%) on seven held-out search benchmarks. Results are grouped by backbone family and search specialization; within each block, + SSP and + SESA are initialized independently from the corresponding unindented backbone. Superscripts on SESA rows show absolute gains over the corresponding base model. The best score within each backbone block is bold.

evaluate three controlled modes: an SSP-trained solver without memory; a SESA-trained solver with the skill bank disabled (SESA-OFF); and the same SESA-trained solver with the final bank enabled (SESA-ON). All three share the same search tool and decoding configuration, and SESA-OFF and SESA-ON use identical weights, so any gap between them comes purely from inference-time retrieval. This design decomposes the benefit of skills into two additive paths: *parametric carryover* (SESA-OFF over SSP), i.e. capability that skill-conditioned self-play leaves inside the policy even after memory is removed; and *retrieval benefit* (SESA-ON over SESA-OFF), i.e. the residual value of the external bank at test time.

Table 2 shows that the parametric path is consistently positive. Relative to SSP, SESA-OFF gains 1.8 points on Qwen3-4B and 2.2 points on Qwen3-8B, even though no skills are retrieved at evaluation time. Re-enabling the same final bank adds another 0.5 and 1.0 points, respectively. The dataset-level effect of retrieval is mixed: relevant guidance can help, but irrelevant context can also distract the solver. Thus most of SESA’s average gain resides in the trained policy, while the bank is best viewed as an optional, model- and task-dependent augmentation.

Mode	NQ	TQA	PQA	HQA	2Wi	MSQ	BBL	Avg.
Qwen3-4B								
SSP (no mem.)	54.4	76.8	53.6	56.4	52.8	25.4	57.6	53.9
SESA-Off	57.8	78.4	56.6	58.2	53.6	25.0	60.0	55.7
SESA-On	56.2	80.4	55.2	57.8	51.8	27.2	64.8	56.2
Qwen3-8B								
SSP (no mem.)	56.0	78.2	55.0	58.0	51.5	28.0	67.2	56.3
SESA-Off	57.8	81.4	57.6	60.2	56.4	31.2	64.8	58.5
SESA-On	62.2	82.8	57.0	64.0	54.6	32.6	63.2	59.5

Table 2: **Dual-path ablation.** SSP uses no memory; SESA-Off and SESA-On share identical weights and differ only in whether the skill bank is enabled at inference. Best per column within each backbone is in bold.

This directly supports our second claim: skill-conditioned self-play is not a prompt-time trick but a training mechanism, since its gains persist in the policy under SESA-OFF, while the retained bank remains available as an optional,

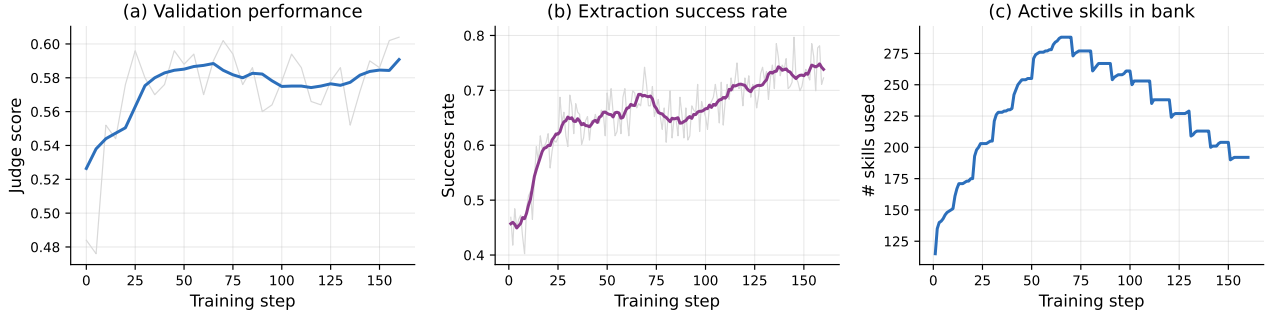


Figure 3: **Self-evolution dynamics during training** (Qwen3-4B). (a) Validation judge score rises early and plateaus at a high level. (b) The challenger’s problem-extraction success rate increases steadily, indicating progressively more solvable, well-formed self-play problems. (c) The count of active skills (retrieved at least once) expands as new skills are distilled and then contracts under deduplication and negative-utility eviction, reflecting a self-refining memory rather than unbounded growth. Light lines are raw per-step values; solid lines are smoothed.

Variant	NQ	TQA	PQA	HQA	2Wi	MSQ	BBL	Avg.
SESA (full)	56.2	80.4	55.2	57.8	51.8	27.2	64.8	56.2
– memory priming	59.2	76.4	56.0	58.0	47.8	26.4	59.2	54.7
– frontier shaping	53.2	76.4	57.2	55.8	51.4	25.6	58.4	54.0
– failure distillation	56.4	75.8	57.2	55.4	50.0	23.8	56.0	53.5

Table 3: **Component leave-one-out** on Qwen3-4B. Each row removes one component while keeping the rest fixed; a larger average drop indicates a more critical component.

lightweight enhancement at deployment.

Component Ablations

Table 3 uses leave-one-out retraining to test memory priming, frontier shaping, and failure distillation. Solver-only access is treated as a design constraint because the stored skills are procedural solving guidance; the supplementary material discusses this choice in detail.

Removing every component lowers the overall average. Without memory priming, performance drops from 56.2 to 54.7, indicating that an initial retrieval and schema anchor remains useful even after online skill growth begins. Removing frontier shaping produces a larger 2.2-point drop, consistent with the need to concentrate learning and consolidation on solvable failures. The largest decrease, 2.7 points, occurs without failure distillation, directly supporting the central claim that persisting lessons adds value beyond self-play updates. Individual datasets vary, but the aggregate ordering identifies failure distillation as the most consequential component in this study.

Training Dynamics

Figure 3 shows that validation quality rises and stabilizes as the challenger produces more usable problems. Meanwhile, active skills first expand and then contract under deduplication and eviction, indicating selective memory refinement rather than unbounded accumulation. Additional definitions

Method	NQ	TQA	PQA	HQA	2Wi	MSQ	BBL	Avg.
Qwen2.5-7B-Instruct	44.2	64.0	36.4	45.0	32.8	16.8	51.2	41.5
+ SSP	54.8	73.4	51.8	51.8	38.8	21.2	54.4	49.5
SkillRL-Search-7B	53.6	69.4	47.8	54.8	45.6	29.2	50.4	50.1
SESA (ours)	57.4	72.2	55.2	52.0	42.2	27.0	51.2	51.0

Table 4: **Comparison with skill-augmented RL** on the Qwen2.5-7B family under a unified evaluation protocol. The best score in each column is bold.

and per-step statistics are provided in the supplementary material.

Evidence for Coupled Evolution

Three complementary observations connect the final gains to the proposed feedback loop. First, SESA-OFF outperforming SSP shows that skill use changes the policy learned during self-play rather than merely adding test-time context. Second, the 2.7-point decrease without failure distillation shows that an evolving bank contributes beyond the initial skills. Third, the dynamics trace simultaneous changes in validation quality, usable problem generation, and active memory. The dynamics alone are correlational, but together with the controlled ablations they support the intended mechanism: self-generated failures alter the skill bank, and the updated bank changes subsequent learning trajectories.

Comparison with Skill-Augmented RL

Because SESA bridges self-play and skill evolution, SSP alone is not a sufficient baseline. We additionally evaluate the released SkillRL-Search-7B (Xia et al. 2026) checkpoint under the same search backend, decoding, and semantic-judging protocol. As shown in Table 4, SkillRL reaches 50.1 average accuracy, exceeding the Qwen2.5-7B-Instruct SSP baseline by 0.6 points. SESA reaches 51.0 under the same protocol and outperforms SkillRL by 0.9 points. This controlled comparison indicates that coupling skill evolution to

an endogenous task frontier provides additional value over fixed-dataset skill learning.

Conclusion

SESA couples self-posed self-play with persistent skill evolution by distilling frontier failures into a maintained memory that changes subsequent training. Across model scales, families, and search-specialized initializations, this closed loop consistently improves average accuracy over SSP, with component ablations identifying online failure distillation as the largest contributor. The Off/On evaluation further shows that skill-conditioned training leaves substantial capability in the model parameters, while the retained bank provides smaller, task-dependent inference gains. SESA thus treats procedural memory as evolving training state rather than an inference-only prompt, supporting both memory-free and memory-augmented deployment.

References

- Acikgoz, E. C.; Qian, C.; Hübotter, J.; Ji, H.; Hakkani-Tür, D.; and Tur, G. 2026. Tool-r0: Self-evolving llm agents for tool-learning from zero data. *arXiv preprint arXiv:2602.21320*.
- Ai, Q.; Fu, Z.; Li, Z.; Jiang, P.; Wu, H.; Song, J.; and He, G. 2026. Cognitive Scaffold: From Fluid Context to Crystallized Memory for Long-Horizon DeepResearch Agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 25526–25542.
- Chen, Y.; Wang, Y.; Zhu, S.; Yu, H.; Feng, T.; Zhang, M.; Patwary, M.; and You, J. 2025. Multi-agent evolve: Llm self-improve through co-evolution. *arXiv preprint arXiv:2510.23595*.
- Grattafiori, A.; Dubey, A.; Jauhri, A.; Pandey, A.; Kadian, A.; Al-Dahle, A.; Letman, A.; Mathur, A.; Schelten, A.; Vaughan, A.; et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Ho, X.; Nguyen, A.-K. D.; Sugawara, S.; and Aizawa, A. 2020. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, 6609–6625.
- Jin, B.; Zeng, H.; Yue, Z.; Yoon, J.; Arik, S. O.; Wang, D.; Zamani, H.; and Han, J. 2025. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. In *Second Conference on Language Modeling*.
- Joshi, M.; Choi, E.; Weld, D. S.; and Zettlemoyer, L. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1601–1611.
- Kwiatkowski, T.; Palomaki, J.; Redfield, O.; Collins, M.; Parikh, A.; Alberti, C.; Epstein, D.; Polosukhin, I.; Devlin, J.; Lee, K.; et al. 2019. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7: 453–466.
- Li, X.; Li, M.; Bao, K.; Ma, Y.; Wang, W.; Liu, D.; and Feng, F. 2026a. SkillGraph: Skill-Augmented Reinforcement Learning for Agents via Evolving Skill Graphs. *arXiv preprint arXiv:2605.12039*.
- Li, Y.; Miao, R.; Qi, Z.; and Lan, T. 2026b. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning. *arXiv preprint arXiv:2603.16060*.
- Lin, H.; Kuai, Z.; Xue, E.; and Wang, L. 2026a. SKILLC: Learning Autonomous Skill Internalization in LLM Agents via Contrastive Credit Assignment. *arXiv preprint arXiv:2605.27899*.
- Lin, H.; Li, P.; Song, J.; Jiang, F.; and Zhang, T. 2026b. Muse-autoskill: Self-evolving agents via skill creation, memory, management, and evaluation. *arXiv preprint arXiv:2605.27366*.
- Lu, H.; Wen, Y.; Cheng, P.; Ding, R.; Guo, J.; Xu, H.; Wang, C.; Chen, H.; xiaoxi jiang; and guanjunjiang. 2026. Search Self-Play: Pushing the Frontier of Agent Capability without Supervision. In *The Fourteenth International Conference on Learning Representations*.
- Mallen, A.; Asai, A.; Zhong, V.; Das, R.; Khashabi, D.; and Hajishirzi, H. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: Long papers)*, 9802–9822.
- Ouyang, S.; Yan, J.; Chen, Y.; Han, R.; Wang, Z.; Mishra, B. D.; Meng, R.; Li, C.-L.; Jiao, Y.; Zha, K.; et al. 2026. Skillos: Learning skill curation for self-evolving agents. *arXiv preprint arXiv:2605.06614*.
- Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N. A.; and Lewis, M. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 5687–5711.
- Pu, H.; Song, X.; and Zhao, L. 2026. SkillOps: Managing LLM Agent Skill Libraries as Self-Maintaining Software Ecosystems. *arXiv preprint arXiv:2605.13716*.
- Qwen; Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; Lin, H.; Yang, J.; Tu, J.; Zhang, J.; Yang, J.; Yang, J.; Zhou, J.; Lin, J.; Dang, K.; Lu, K.; Bao, K.; Yang, K.; Yu, L.; Li, M.; Xue, M.; Zhang, P.; Zhu, Q.; Men, R.; Lin, R.; Li, T.; Tang, T.; Xia, T.; Ren, X.; Ren, X.; Fan, Y.; Su, Y.; Zhang, Y.; Wan, Y.; Liu, Y.; Cui, Z.; Zhang, Z.; and Qiu, Z. 2025. Qwen2.5 Technical Report. *arXiv:2412.15115*.
- Shi, Y.; Chen, Y.; Lu, Z.; Miao, Y.; Liu, S.; Gu, Q.; Cai, X.; Wang, X.; and Zhang, A. 2026. Skill1: Unified evolution of skill-augmented agents via reinforcement learning. *arXiv preprint arXiv:2605.06130*.
- Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36: 8634–8652.
- Trivedi, H.; Balasubramanian, N.; Khot, T.; and Sabharwal, A. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics*, 10: 539–554.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*.

Wang, J.; Yan, Q.; Wang, Y.; Tian, Y.; Mishra, S. S.; Xu, Z.; Gandhi, M.; Xu, P.; and Cheong, L. L. 2026. Reinforcement learning for self-improving agent with skill library. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 1529–1550.

Wu, R.; Wang, X.; Mei, J.; Cai, P.; Fu, D.; Yang, C.; Wen, L.; Yang, X.; Shen, Y.; Wang, Y.; et al. 2025. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint arXiv:2510.16079*.

Xia, P.; Chen, J.; Wang, H.; Liu, J.; Zeng, K.; Wang, Y.; Han, S.; Zhou, Y.; Zhao, X.; Chen, H.; et al. 2026. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*.

Xia, P.; Zeng, K.; Liu, J.; Qin, C.; Wu, F.; Zhou, Y.; Xiong, C.; and Yao, H. 2025. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint arXiv:2511.16043*.

Yang, A.; Li, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Gao, C.; Huang, C.; Lv, C.; et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W.; Salakhutdinov, R.; and Manning, C. D. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2369–2380.

Yu, W.; Liang, Z.; Huang, C.; Panaganti, K.; Fang, T.; Mi, H.; and Yu, D. 2025. Guided self-evolving llms with minimal human supervision. *arXiv preprint arXiv:2512.02472*.

Zhang, H.; Fan, S.; Zou, H. P.; Chen, Y.; Wang, Z.; Zhou, J.; Li, C.; Huang, W.-C.; Yao, Y.; Zheng, K.; et al. 2026. Coevoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*.

Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, 19632–19642.